

**ENVIROSCAN: AI-POWERED
POLLUTION SOURCE IDENTIFIER
USING
GEOSPATIAL ANALYTICS**

CONTENTS

ABSTARCT.....	3
PROBLEM STATEMENT	4
OBJECTIVE	5
SCOPE OF THE PROJECT	6
INTRODUCTION	7
SYSTEM ARCHITECTURE	8
DATASET DESCRIPTION	9
SOFTWARE REQUIREMENTS.....	10
HARDWARE REQUIREMENTS.....	11
TOOLS AND TECHNOLOGIES.....	12
MILESTONES AND MODULES	
1.1 MODULE- 1.....	13
1.2 MODULE- 2.....	16
1.3 MODULE- 3.....	19
DATA ANALYSIS USING PLOTS.....	22
1.4 MODULE- 4.....	27
RESULTS.....	30
CONCLUSION.....	31
FUTURE SCOPE.....	32
REFERENCES	33

ABSTRACT

This project proposes an intelligent pollution monitoring system called **EnviroScan**, which uses machine learning and geospatial analytics to identify the likely sources of air pollution.

Traditional pollution monitoring systems measure pollutant levels but often fail to determine the source of pollution.

The system integrates **pollution data, weather parameters, and geographical information** to predict pollution sources such as vehicular traffic, industrial activity, agricultural burning, and natural causes.

Machine learning models such as **Random Forest, Decision Tree, and XGBoost** are trained using pollution and environmental data.

The predictions are visualized using **interactive geospatial heatmaps**, and alerts are generated when pollution exceeds safe thresholds.

The final system includes a **dashboard for real-time monitoring and visualization**, enabling environmental agencies and urban planners to make informed decisions.

PROBLEM STATEMENT

Air pollution is one of the most critical environmental challenges affecting urban and rural areas worldwide. Traditional air quality monitoring systems focus on measuring pollutant concentrations such as PM_{2.5}, PM₁₀, NO₂, CO, and SO₂.

However, these systems do not identify the **specific source of pollution**, making it difficult for authorities to implement targeted mitigation strategies.

Without accurate source identification, it becomes challenging to determine whether pollution originates from **vehicular traffic, industrial emissions, agricultural activities, or natural causes**.

The **EnviroScan system** addresses this limitation by integrating pollution sensor data, weather information, and geospatial features to **predict the most likely source of pollution using machine learning techniques**.

OBJECTIVES

- Collect real-time air pollution data from external APIs.
- Integrate weather data and geographical location features.
- Perform data pre-processing and feature engineering.
- Train machine learning models to classify pollution sources.
- Evaluate model performance using standard metrics.
- Visualize pollution patterns using geospatial heatmaps.
- Develop an interactive dashboard for monitoring pollution levels.
- Generate alerts for high pollution zones.

SCOPE OF THE PROJECT

The scope of this project includes analysing environmental data to identify pollution sources using machine learning models.

The system can be used by environmental agencies, urban planners, and researchers to understand pollution patterns and identify high-risk areas.

The project focuses on integrating multiple data sources, including pollution monitoring APIs, weather data, and geographic information systems.

The system also provides visualization tools that allow users to view pollution hotspots and source predictions on interactive maps.

Although the project demonstrates the concept using selected locations, the system can be extended to support real-time pollution monitoring across multiple cities.

INTRODUCTION

Air pollution is one of the most critical environmental problems affecting modern cities and rural areas. Rapid industrialization, urbanization, and increased vehicular traffic have significantly contributed to the deterioration of air quality.

Pollutants such as particulate matter (PM_{2.5} and PM₁₀), nitrogen dioxide (NO₂), sulphur dioxide (SO₂), and carbon monoxide (CO) are harmful to human health and can cause respiratory diseases, cardiovascular problems, and environmental damage.

While many countries have implemented air quality monitoring systems, these systems primarily measure pollutant concentrations and provide air quality indices. However, they often fail to identify the **specific sources of pollution**, making it difficult to implement targeted pollution control measures.

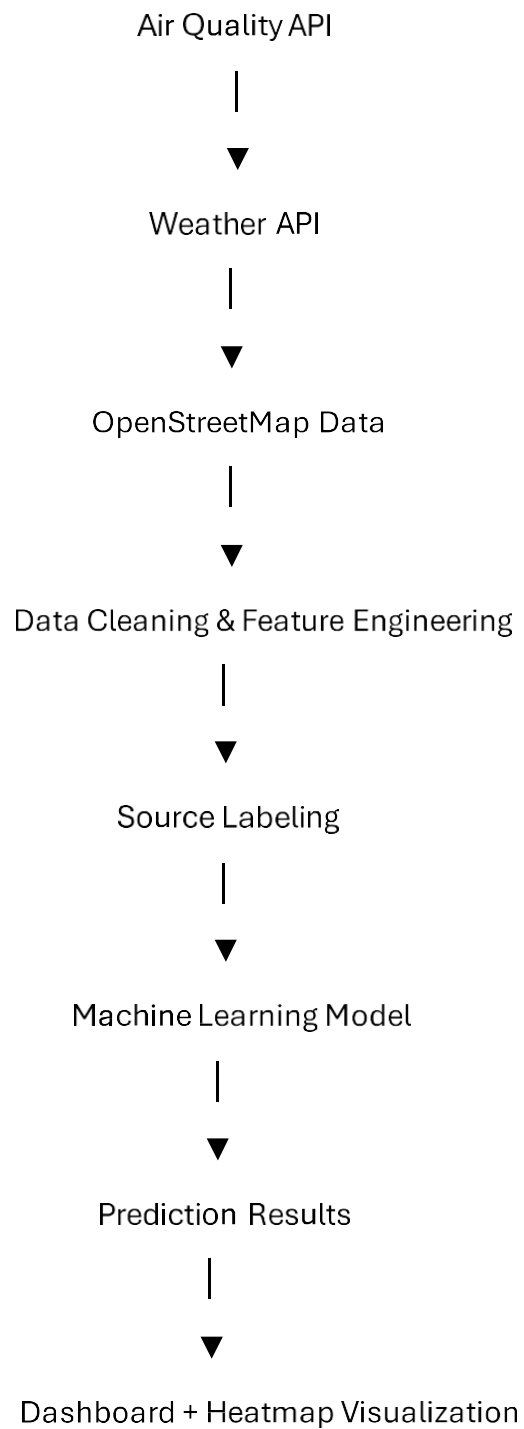
Advancements in **machine learning and geospatial analytics** provide an opportunity to analyse environmental data more effectively. By combining pollution sensor data, weather conditions, and geographic information, machine learning models can identify patterns and predict the likely sources of pollution.

The **EnviroScan project** aims to develop an intelligent pollution monitoring system capable of identifying pollution sources and visualizing pollution patterns through interactive dashboards and maps.

For this project, pollution data analysis was performed for two areas in Vijayawada: HB Colony and Kanuru. HB Colony is primarily a residential region with moderate traffic activity, while Kanuru is an area that includes residential zones as well as nearby industrial and commercial activities.

Studying these two locations helps analyse how different environmental and human activities influence pollution levels in urban areas.

SYSTEM ARCHITECTURE



DATASET DESCRIPTION

The dataset used in the EnviroScan project was created by combining multiple environmental data sources, including air quality data, weather data, and geographical features. The data was collected for selected monitoring stations in **Vijayawada**, particularly the areas of HB Colony and Kanuru. The final dataset contains **1914 records and 33 columns**.

The air quality dataset contains measurements of major pollutants such as particulate matter (PM_{2.5} and PM₁₀), nitrogen dioxide (NO₂), sulphur dioxide (SO₂), carbon monoxide (CO), and ozone (O₃). These pollutants are commonly used to assess air pollution levels and their impact on human health.

Weather data was also incorporated into the dataset to understand the influence of atmospheric conditions on pollution levels. Weather parameters include temperature, humidity, wind speed, and rainfall. These factors play an important role in the dispersion and accumulation of pollutants in the atmosphere.

In addition to pollutant and weather data, geographical and environmental features were added to the dataset. These features include road density, industrial area counts, farmland counts, and waste site counts near each monitoring location. These attributes help identify possible pollution sources such as vehicular emissions, industrial activities, agricultural practices, and waste burning.

The final dataset consists of multiple features representing **environmental conditions, pollutant concentrations, and location-based factors**, which were used to perform data analysis and train machine learning models for pollution source classification.

Dataset Features

The dataset contains the following major features:

Timestamp, Location ID, Location Name, Latitude, Longitude, PM_{2.5}, PM₁₀, NO₂, SO₂, CO, O₃, Temperature, Humidity, Wind Speed, Wind Direction, Pressure, Road Count, Industrial Count, Farmland Count, Waste Count, Time-based features (Hour, Day of Week, Month, Season), Spatial Features

SOFTWARE REQUIREMENTS

The **EnviroScan project** was developed using a system running **Windows 11** as the operating system, which provides the platform for running all development tools and executing the project.

The programming and development of the project were carried out using **Visual Studio Code**, which is a lightweight and powerful code editor used for writing, editing, and managing the project files and Python scripts.

The project uses **Python** as the main programming language for performing data preprocessing, analysis, visualization, and machine learning tasks.

To run Python programs and manage packages, the **Windows Command Prompt** was used, which allows executing Python scripts and installing required libraries through commands.

These software components together provide the necessary environment for developing, running, and testing the EnviroScan air pollution analysis system.

HARDWARE REQUIREMENTS

The hardware requirements for the **EnviroScan project** are minimal because the system is developed using lightweight data analysis and machine learning techniques.

A computer with a modern processor such as **Intel Core i5** or an equivalent processor is sufficient to run the project smoothly. The system should have a minimum of **8 GB RAM** to handle data processing, visualization, and model training efficiently.

Additionally, at least **256 GB of storage** is recommended to store the dataset, project files, and required software packages.

An **internet connection** is also necessary for downloading required libraries, accessing environmental datasets, and performing updates during the development process.

These hardware components provide an adequate environment for implementing and running the EnviroScan air pollution monitoring and analysis system.

TOOLS AND TECHNOLOGIES

The **EnviroScan project** uses several tools and technologies for data collection, processing, analysis, machine learning modelling, and visualization. The primary programming language used in the project is **Python**, which provides powerful libraries for data analysis and machine learning applications.

Environmental and pollution data can be obtained through open data platforms such as **OpenAQ** for air quality information and **OpenWeatherMap** for weather-related data. Geographic and location-based features are extracted from **OpenStreetMap** using the **OSMnx** library, which helps identify nearby roads, industrial areas, farmland, and waste locations around monitoring stations.

For data pre-processing and analysis, libraries such as **Pandas** and **NumPy** are used to clean, organize, and manipulate the dataset efficiently. Data visualization is performed using **Matplotlib** and **Seaborn**, which help generate graphs such as pollutant distribution plots, correlation heatmaps, and time-based trend charts. Interactive geographic visualizations and pollution maps can also be created using **Folium**.

For machine learning modelling, algorithms such as **Decision Tree**, **Random Forest** and **XGBoost** are used to classify the probable source of pollution based on pollutant concentrations, weather conditions, and surrounding environmental features.

These models are trained using the **Scikit-learn** framework, which provides tools for model training, evaluation, and prediction. The trained models are then saved using **Joblib**, which allows the models to be stored and reused later without retraining.

The project interface can be developed using **Streamlit**, which enables the creation of an interactive web dashboard where users can visualize pollution data, run predictions, and explore analysis results.

For version control and project sharing, the complete project code and files are stored and managed using **GitHub**, which allows easy collaboration, code tracking, and deployment. Together, these tools and technologies provide a complete environment for building the EnviroScan system, performing environmental data analysis, training machine learning models, and presenting results through an interactive platform.

MILESTONES AND MODULES

The milestones and modules section contains four milestones each milestone contains two modules.

Milestone-1: Data collection, Data cleaning and feature Engineering

Milestone-1 focuses on gathering environmental datasets from multiple sources and preparing the data for machine learning analysis. The collected raw data may contain inconsistencies, missing values, and redundant information, which must be cleaned and processed before building predictive models.

This milestone includes **two modules** that handle data acquisition and data pre-processing tasks.

1.1 MODULE-1: Data Collection

Module-1 focuses on collecting environmental data from various sources such as pollution monitoring APIs, weather APIs, and geographical databases. The objective of this module is to gather relevant information that can help identify possible pollution sources.

Air pollution data is collected using environmental monitoring APIs such as OpenAQ. This dataset contains measurements of important pollutants including PM2.5, PM10, nitrogen dioxide (NO₂), carbon monoxide (CO), sulphur dioxide (SO₂), and ozone (O₃). Each data record includes pollutant concentration levels along with location coordinates and timestamps.

Weather data is collected using the OpenWeatherMap API. Weather conditions play an important role in the spread and dispersion of air pollutants. The weather parameters collected include temperature, humidity, pressure, wind speed, and wind direction.

In addition to pollution and weather data, geographic features around each monitoring location are extracted using OpenStreetMap. Libraries such as OSMnx are used to identify nearby roads, industrial areas, farmland regions, and waste disposal sites. These location features help determine the potential source of pollution in a given area.

All collected datasets are stored in structured formats such as CSV files or JSON files. Each record contains important attributes including location coordinates, pollutant levels, weather parameters, and nearby environmental features.

IMPLEMENTATION:

Air Quality Data Collection Module

The first script is responsible for collecting **hourly air pollution data** from the **OpenAQ API** for monitoring stations located in **Vijayawada**. At the beginning of the script, the API key, base URL, and request headers are defined. These values are required to authenticate requests and retrieve air quality data from the OpenAQ platform. The monitoring stations are also defined in a list containing: Station ID, Station name, Latitude, Longitude. Two monitoring stations are used: HB Colony, Kanuru. Each monitoring station can contain multiple sensors measuring different pollutants. The code sends a request to the OpenAQ API to retrieve all sensors available at each station.

The script then filters sensors to select only the required pollutants:

- **PM2.5**
- **PM10**
- **NO₂**
- **SO₂**
- **CO**
- **O₃**.

To improve efficiency, the script uses **parallel processing** with `ThreadPoolExecutor`. Each pollutant sensor is processed in a separate thread, allowing multiple API requests to run simultaneously. This significantly reduces the time required to download large amounts of hourly data.

2. Weather Data Collection Module

The second script collects weather data that influences pollution levels. Weather information is retrieved using **OpenWeatherMap API**

For each monitoring station, the script performs the following steps. The script queries the API to retrieve hourly weather data between the selected dates. The following meteorological variables are collected

- **Temperature**
- **Relative humidity**
- **Surface pressure**
- **Wind speed**
- **Wind direction.**

3. Geographic Feature Extraction Module

The third script extracts environmental and geographical features around each monitoring station using data from **OpenStreetMap** through the **OSMnx** library.

These geographic features help identify potential sources of pollution. A **5 km radius** is defined around each monitoring station. The script searches within this radius to identify nearby environmental features. The script uses the road network graph to count the number of road segments within the radius. High road density often indicates higher vehicle emissions. The system identifies land areas labelled as industrial zones in OpenStreetMap. These regions may contribute to pollution through industrial emissions. Waste-related features such as landfills and waste disposal locations are extracted. Waste burning or landfill gases can influence air quality. The script also detects farmland and agricultural zones, which may contribute to pollution through crop burning or fertilizer use.

The geographic feature dataset contains:

- **Road count**
- **Industrial area count**
- **Waste site count**
- **Farmland count**

for each monitoring station.

The final script combines all datasets into a single integrated dataset used for analysis and machine learning. The script loads three datasets: Air quality dataset, Weather dataset, Geographic features dataset. Since each dataset may store timestamps in different formats, the script converts all timestamps into a consistent datetime format. The timestamps are then rounded to hourly intervals to ensure proper alignment during merging.

The final dataset includes: **Pollution Variables, Weather Variables, Geographic Variables**

Output:

POSSIBLE DATA LOSS Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, save it in an Excel file format.																					Don't show again		Save As...	
city																								
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V			
city	timestamp	location_i	location	lat	lon	pm2_5	pm10	no2	o3	so2	co	temperature	humidity	pressure	f_wind	speed	wind_dir	road_cour	industrial_waste_coi	farmland_count				
Vijayawada	01-01-2026 06:00	3409492 HB Colony	16.5206	80.6404	105	189	13.6	24.3	10.1	0.64	20.6	96	1009.7	1.68	63	37888	15	3	180					
Vijayawada	01-01-2026 07:00	3409492 HB Colony	16.5206	80.6404	88.4	158	13.5	24.3	10.1	0.64	21.5	91	1010.6	1.77	50	37888	15	3	180					
Vijayawada	01-01-2026 08:00	3409492 HB Colony	16.5206	80.6404	79.8	146	13.5	24.3	8.69	0.645	23.8	80	1011.6	2.16	77	37888	15	3	180					
Vijayawada	01-01-2026 09:00	3409492 HB Colony	16.5206	80.6404	52.8	96.9	13.8	26	8.22	0.615	26	67	1012	2.77	97	37888	15	3	180					
Vijayawada	01-01-2026 10:00	3409492 HB Colony	16.5206	80.6404	62.2	116	16.5	17.8	10.4	0.4	28.2	56	1011.8	2.57	97	37888	15	3	180					
Vijayawada	01-01-2026 11:00	3409492 HB Colony	16.5206	80.6404	72.4	142	16.5	22.1	9.81	0.4	29	52	1011.4	2.15	89	37888	15	3	180					
Vijayawada	01-01-2026 12:00	3409492 HB Colony	16.5206	80.6404	47	84.3	17.4	24.2	9.14	0.525	29.9	48	1010.2	1.81	84	37888	15	3	180					
Vijayawada	01-01-2026 13:00	3409492 HB Colony	16.5206	80.6404	54.5	91.8	18	28.5	13	0.688	30.7	44	1008.7	1.88	79	37888	15	3	180					
Vijayawada	01-01-2026 14:00	3409492 HB Colony	16.5206	80.6404	51.1	84.2	23.1	30.7	10.4	0.8	31	41	1007.6	2.11	68	37888	15	3	180					
Vijayawada	01-01-2026 15:00	3409492 HB Colony	16.5206	80.6404	34	53.8	20.1	33.8	13.6	0.725	30.9	41	1007.2	2.13	66	37888	15	3	180					
Vijayawada	01-01-2026 16:00	3409492 HB Colony	16.5206	80.6404	38.6	59.6	18.4	30.9	14.3	0.683	30.3	42	1007.4	1.9	72	37888	15	3	180					
Vijayawada	01-01-2026 17:00	3409492 HB Colony	16.5206	80.6404	33.9	52.2	13.3	32.9	14	0.88	28.4	49	1007.8	2.06	104	37888	15	3	180					
Vijayawada	01-01-2026 18:00	3409492 HB Colony	16.5206	80.6404	35.5	54.7	11.6	34.3	8.08	0.665	25.8	68	1008.7	3.54	134	37888	15	3	180					
Vijayawada	01-01-2026 19:00	3409492 HB Colony	16.5206	80.6404	42.4	65.4	11.5	33.5	7.92	0.66	24.6	75	1009.8	2.59	124	37888	15	3	180					
Vijayawada	01-01-2026 20:00	3409492 HB Colony	16.5206	80.6404	53.4	82.6	11.5	29.3	7.92	0.66	24	78	1010.9	2.44	125	37888	15	3	180					
Vijayawada	01-01-2026 21:00	3409492 HB Colony	16.5206	80.6404	43.7	67.6	11.5	31.3	7.95	0.66	23.3	83	1011.3	2.02	120	37888	15	3	180					
Vijayawada	01-01-2026 22:00	3409492 HB Colony	16.5206	80.6404	35.4	54.7	11.5	29.3	7.91	0.66	22.8	86	1011.4	1.36	107	37888	15	3	180					
Vijayawada	01-01-2026 23:00	3409492 HB Colony	16.5206	80.6404	50.4	77.8	11.5	23.4	7.9	0.66	22.5	91	1011.3	1.25	106	37888	15	3	180					
Vijayawada	02-01-2026 00:00	3409492 HB Colony	16.5206	80.6404	30	42.1	12	33.7	4.91	0.66	22.1	93	1011	1.25	88	37888	15	3	180					
Vijayawada	02-01-2026 01:00	3409492 HB Colony	16.5206	80.6404	29.6	41.4	12	33.7	4.89	0.66	21.8	94	1010	1.61	64	37888	15	3	180					
Vijayawada	02-01-2026 02:00	3409492 HB Colony	16.5206	80.6404	29.5	41.2	12	33.7	4.87	0.66	21.5	94	1009.9	1.52	49	37888	15	3	180					
Vijayawada	02-01-2026 03:00	3409492 HB Colony	16.5206	80.6404	29.5	41.1	12	33.7	4.86	0.66	21.2	92	1009.9	2.02	42	37888	15	3	180					
Vijayawada	02-01-2026 04:00	3409492 HB Colony	16.5206	80.6404	29.5	41.1	12	33.7	4.89	0.66	20.8	92	1010.1	2.16	40	37888	15	3	180					
Vijayawada	02-01-2026 05:00	3409492 HB Colony	16.5206	80.6404	51.7	83.4	12	33.7	4.89	0.675	21.4	89	1012.4	2.19	63	37888	15	3	180					
Vijayawada	02-01-2026 06:00	3409492 HB Colony	16.5206	80.6404	36.8	55	12	33.7	4.88	0.67	23.6	79	1013.6	2.44	79	37888	15	3	180					
Vijayawada	02-01-2026 07:00	3409492 HB Colony	16.5206	80.6404	51.5	83	12	33.7	4.89	0.668	25.9	68	1013.9	2.6	92	37888	15	3	180					
vijayawada final dataset																								

MODULE-2: Data Cleaning and Feature Engineering

After collecting the raw datasets, the next step is to clean and pre-process the data to ensure accuracy and consistency. Raw environmental data may contain missing values, duplicate entries, and inconsistent formats that must be corrected before using the data for machine learning.

IMPLEMENTATION:

The dataset `vijayawada_final_dataset.csv` is loaded using the Pandas library. Basic information such as dataset shape, column names, and sample rows are displayed to understand the structure of the data. The dataset initially contains: **1914 records, 21 columns** including pollutant levels, weather variables, and geographic attributes. Duplicate records are removed to ensure data integrity. Pollutant columns such as **PM2.5, PM10, NO₂, SO₂, CO, and O₃** are converted into numeric format. If any pollutant value is **negative**, it is considered invalid and replaced with **NaN (missing value)**. This step ensures the pollution data used for modelling is valid and consistent.

The dataset is then sorted by location ID and timestamp to maintain chronological order of environmental measurements. Missing values are treated using a three-level strategy: **Linear interpolation** within each monitoring station to estimate nearby missing values. Remaining missing values are filled using the **mean value of the station**. If any values still remain missing, they are replaced with the **global dataset mean**. This approach maintains data continuity while minimizing information loss.

To prepare the dataset for machine learning algorithms, numerical features are scaled using **Scikit-learn, MinMaxScaler**. Normalization converts values into a **0–1 range**, allowing different variables such as pollutant concentrations, weather data, and land-use counts to be compared effectively. Normalized variables include, Air pollutants (PM2.5, PM10, NO₂, SO₂, CO, O₃), Weather variables (temperature, humidity, pressure, wind speed, wind direction), Geographic counts (roads, industries, waste sites, farmland).

To understand how pollution is influenced by nearby infrastructure, spatial distance features are calculated using the **Haversine formula**, which measures geographic distance between two coordinates. Distances are calculated from each monitoring point to City centre, Industrial area, Major Road network, Waste dumping site. These spatial features help models learn how pollution levels change depending on proximity to pollution sources.

Several time-based features are extracted from the timestamp:

- **Hour** – time of the day
- **Day of week** – weekday index
- **Day of year** – seasonal progression
- **Month** – month number

Additional logical indicators are created:

- **Weekend indicator**
- **Rush hour indicator (7–10 AM, 5–8 PM)**

- **Dry season indicator**

A seasonal classification is also generated:

- **Winter**
- **Summer**
- **Monsoon**
- **Post-Monsoon**

These features allow the model to capture **daily and seasonal pollution patterns**. These features improve predictive modelling by capturing real-world environmental dynamics. After feature engineering, the dataset is validated by displaying: Dataset information, Statistical summary, Sample preview of records. The final processed dataset contains: **1914 rows, 36 columns**

Output:

L1610		0.180517616556322																																	
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T															
1	city	timestamp	location_i	location	lat	lon	pm2_5	pm10	no2	o3	so2	co	temperatu	humidity	pressure_hj	wind_speed	wind_direction	road_cour	industrial_waste_count	f															
2	Vijayawada	01-01-2026 06:00	3409392	Kanuru, Vijayav	16.489	80.694	0.025761	0.008536	0.083871	0.235855	0.014342	0.273114	0.103627	0.949367	0.465409	0.3302239	0.175977654	0	0	1															
3	Vijayawada	01-01-2026 07:00	3409392	Kanuru, Vijayav	16.489	80.694	0.023754	0.007051	0.142158	0.235855	0.014473	0.273114	0.145078	0.886076	0.522013	0.3414179	0.148044693	0	0	1															
4	Vijayawada	01-01-2026 08:00	3409392	Kanuru, Vijayav	16.489	80.694	0.024757	0.007608	0.156396	0.235855	0.015126	0.280059	0.259067	0.746835	0.591195	0.4141791	0.206703911	0	0	1															
5	Vijayawada	01-01-2026 09:00	3409392	Kanuru, Vijayav	16.489	80.694	0.032118	0.013732	0.085651	0.235855	0.015779	0.277744	0.378238	0.594937	0.616352	0.5541045	0.262569832	0	0	1															
6	Vijayawada	01-01-2026 10:00	3409392	Kanuru, Vijayav	16.489	80.694	0.057879	0.035257	0.242269	0.235855	0.015256	0.263855	0.487047	0.455696	0.591195	0.505597	0.25698324	0	0	1															
7	Vijayawada	01-01-2026 11:00	3409392	Kanuru, Vijayav	16.489	80.694	0.307461	0.241047	1	0.235855	0.018783	0.243021	0.528497	0.417722	0.572327	0.4477612	0.251396648	0	0	1															
8	Vijayawada	01-01-2026 12:00	3409392	Kanuru, Vijayav	16.489	80.694	0.769154	0.628874	0.178198	0.235855	0.018783	0.240706	0.569948	0.379747	0.503145	0.363806	0.245810056	0	0	1															
9	Vijayawada	01-01-2026 13:00	3409392	Kanuru, Vijayav	16.489	80.694	0.093008	0.046947	0.096774	0.235855	0.020089	0.233761	0.606218	0.329114	0.408805	0.3731343	0.254189944	0	0	1															
10	Vijayawada	01-01-2026 14:00	3409392	Kanuru, Vijayav	16.489	80.694	0.169957	0.091668	0.086541	0.244225	0.019828	0.243021	0.632124	0.291139	0.333333	0.3843284	0.229050279	0	0	1															
11	Vijayawada	01-01-2026 15:00	3409392	Kanuru, Vijayav	16.489	80.694	0.287387	0.174244	0.094105	0.250921	0.020089	0.249965	0.626943	0.278481	0.314465	0.4048507	0.203910615	0	0	1															
12	Vijayawada	01-01-2026 16:00	3409392	Kanuru, Vijayav	16.489	80.694	0.21278	0.157543	0.085651	0.272682	0.022571	0.24765	0.595855	0.278481	0.327044	0.4197761	0.234636872	0	0	1															
13	Vijayawada	01-01-2026 17:00	3409392	Kanuru, Vijayav	16.489	80.694	0.52827	0.32455	0.083871	0.26766	0.020481	0.254595	0.455959	0.468354	0.352201	0.6324627	0.349162011	0	0	1															
14	Vijayawada	01-01-2026 18:00	3409392	Kanuru, Vijayav	16.489	80.694	0.117096	0.119132	0.087875	0.24841	0.019828	0.26154	0.352332	0.620253	0.408805	0.6492537	0.354748603	0	0	1															
15	Vijayawada	01-01-2026 19:00	3409392	Kanuru, Vijayav	16.489	80.694	0.116092	0.117647	0.083871	0.24841	0.02022	0.298579	0.316062	0.670886	0.471698	0.5466418	0.349162011	0	0	1															
16	Vijayawada	01-01-2026 20:00	3409392	Kanuru, Vijayav	16.489	80.694	0.112078	0.113194	0.082536	0.24841	0.018261	0.305523	0.26943	0.734177	0.540881	0.4701493	0.360335196	0	0	1															
17	Vijayawada	01-01-2026 21:00	3409392	Kanuru, Vijayav	16.489	80.694	0.092339	0.090555	0.083426	0.24841	0.018261	0.312468	0.238342	0.78481	0.566038	0.363806	0.326815642	0	0	1															
18	Vijayawada	01-01-2026 22:00	3409392	Kanuru, Vijayav	16.489	80.694	0.068585	0.063648	0.082536	0.24841	0.017477	0.307838	0.212435	0.822785	0.572327	0.2723881	0.351955307	0	0	1															
19	Vijayawada	01-01-2026 23:00	3409392	Kanuru, Vijayav	16.489	80.694	0.065239	0.059937	0.084316	0.24841	0.01813	0.303208	0.196891	0.873418	0.566038	0.2518657	0.287709497	0	0	1															
20	Vijayawada	02-01-2026 00:00	3409392	Kanuru, Vijayav	16.489	80.694	0.069923	0.065318	0.083426	0.24841	0.017869	0.291634	0.15544	0.860759	0.742138	0.2537313	0.262569832	0	0	1															
21	Vijayawada	02-01-2026 01:00	3409392	Kanuru, Vijayav	16.489	80.694	0.039478	0.030618	0.085651	0.24841	0.013428	0.284689	0.124352	0.898734	0.704403	0.2052239	0.259776536	0	0	1															
22	Vijayawada	02-01-2026 02:00	3409392	Kanuru, Vijayav	16.489	80.694	0.044496	0.03637	0.086096	0.24841	0.01395	0.289319	0.093264	0.936709	0.679245	0.1865672	0.142458101	0	0	1															
23	Vijayawada	02-01-2026 03:00	3409392	Kanuru, Vijayav	16.489	80.694	0.048511	0.040824	0.083871	0.24841	0.01382	0.307838	0.082902	0.936709	0.679245	0.1809701	0.092178771	0	0	1															
24	Vijayawada	02-01-2026 04:00	3409392	Kanuru, Vijayav	16.489	80.694	0.039813	0.030989	0.085206	0.24841	0.01395	0.307838	0.088083	0.924051	0.672956	0.261194	0.092178771	0	0	1															
25	Vijayawada	02-01-2026 07:00	3409392	Kanuru, Vijayav	16.489	80.694	0.04918	0.041752	0.083426	0.24841	0.014211	0.289319	0.124352	0.860759	0.811321	0.2649254	0.022346369	0	0	1															
26	Vijayawada	02-01-2026 08:00	3409392	Kanuru, Vijayav	16.489	80.694	0.044831	0.036742	0.083426	0.24841	0.014211	0.293949	0.243523	0.734177	0.861635	0.2910448	0.120111732	0	0	1															
27	Vijayawada	02-01-2026 09:00	3409392	Kanuru, Vijayav	16.489	80.694	0.035798	0.02635	0.083426	0.24841	0.014473	0.277744	0.367876	0.544304	0.893082	0.5223881	0.237430168	0	0	1															

U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH
farmland_count	dist_to_center	dist_to_industry	dist_to_road	dist_to_dump	hour	day_of_week	day_of_year	month	is_weekend	is_rush_hour	is_dry_season	season	
1	5.264115926	4.848066027	5.57487642	3.68597257	6	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	7	3	1	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	8	3	1	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	9	3	1	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	10	3	1	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	11	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	12	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	13	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	14	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	15	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	16	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	17	3	1	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	18	3	1	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	19	3	1	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	20	3	1	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	21	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	22	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	23	3	1	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	0	4	2	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	1	4	2	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	2	4	2	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	3	4	2	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	4	4	2	1	0	0	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	7	4	2	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	8	4	2	1	0	1	1	1	
1	5.264115926	4.848066027	5.57487642	3.68597257	9	4	2	1	0	1	1	1	

MILESTONE-2: Pollution Source Labelling and Model Training

This milestone focuses on identifying the **major sources of air pollution** by applying rule-based labelling techniques and building a machine learning model for pollution source prediction. The processed dataset from Milestone-1 was used as input for this stage.

MODULE-3: Pollution Source Labelling

Air pollution can originate from multiple sources such as **vehicular emissions, industrial activities, agricultural practices, and waste burning**. To identify the dominant source of pollution for each data point, a **rule-based labelling approach** was implemented.

This approach uses pollutant concentration levels and nearby environmental indicators such as **road density, industrial areas, farmland presence, and waste sites** to determine the likely pollution source.

Pollution Source Categories

The following categories were defined:

Pollution Source Description

Vehicular	Pollution caused by traffic emissions.
Industrial	Pollution caused by factories and industrial activities
Agricultural	Pollution caused by dust, crop burning, and farming activities.
Burning	Pollution caused by waste burning or open garbage burning.
Natural	Pollution caused by natural environmental factors

Implementation:

This script assigns pollution source labels to the processed environmental dataset collected from monitoring stations in **Vijayawada**. The goal is to classify each record according to the most likely pollution source using rule-based logic. Load Dataset The feature-engineered dataset `vijayawada_feature_engineering.csv` is loaded using **Pandas**. The timestamp column is converted into datetime format to enable time-based feature extraction. Create Seasonal Indicators, Two additional columns are created from the timestamp: **month** extracts the month from the timestamp. **is_dry** – identifies dry season months (November, December, January, February) This helps detect pollution caused by seasonal agricultural activities such as crop burning

Calculate Dynamic Thresholds, instead of fixed limits, the script calculates **dynamic thresholds using percentiles** of the dataset. 75th percentile for pollutants such as **NO₂, SO₂, PM2.5, PM10, CO, O₃**, 75th percentile for **road density**, 50th percentile for **industrial density**, 70th percentile for **farmland and waste areas**. These thresholds identify **high pollution levels relative to the dataset distribution**. A function `assign_label()` assigns pollution source categories based on environmental conditions and pollutant levels. The classification rules are:

Vehicular Pollution

- High road density
- Rush hour time (9–12 AM or 5–8 PM)
- High **NO₂** or **CO**

Industrial Pollution

- High industrial area count
- High **SO₂** or **PM2.5**

Agricultural Pollution

- Near farmland
- Dry season
- High **PM10** or **PM2.5**

Burning Pollution

- Near waste sites
- High **CO** or **NO₂** or **PM2.5**
- Often associated with garbage burning

Natural Pollution

- Occurs when pollutant levels are low
- Typically dominated by natural atmospheric processes such as ozone formation.

If none of the rules match clearly, a **fallback logic** selects the pollution source based on the dominant pollutant combination. The `assign_label()` function is applied to every record in the dataset to generate a new column:

Each record is labelled as one of the following classes:

- Vehicular
- Industrial
- Agricultural
- Burning
- Natural

The script prints statistics showing Total count of each pollution source, Distribution of pollution types per monitoring station. This helps verify whether the labelling is balanced across locations. A final dataset is created containing: Location information, Pollution measurements, Weather variables, Geographic features, Temporal features, Pollution source label. The dataset is sorted by **location and timestamp** to maintain chronological order. This dataset is used to train machine learning models such as **Decision Tree** and **XGBoost**, and the trained models can be saved using **Joblib** for later prediction in the EnviroScan dashboard.

Output:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
1	city	timestamp	location_id	location	lat	lon	pm2_5	pm10	no2	o3	so2	co	temperatu	humidity	pressure_t	wind_speed	wind_dir	road_cour	industrial	waste_cot	farmland	pollution_sour
2	Vijayawada	01-01-2026 06:00	3409492	HB Colony	16.5206	80.6404	0.294078	0.293004	0.023804	0.19066	0.008725	0.145794	0.093264	0.949367	0.427673	0.313433	0.170391	1	1	0	0	Industrial
3	Vijayawada	01-01-2026 07:00	3409492	HB Colony	16.5206	80.6404	0.238541	0.23548	0.023359	0.19066	0.008725	0.145794	0.139896	0.886076	0.484277	0.330224	0.134078	1	1	0	0	Industrial
4	Vijayawada	01-01-2026 08:00	3409492	HB Colony	16.5206	80.6404	0.209769	0.213212	0.023359	0.19066	0.006884	0.146951	0.259067	0.746835	0.54717	0.402985	0.209497	1	1	0	0	Industrial
5	Vijayawada	01-01-2026 09:00	3409492	HB Colony	16.5206	80.6404	0.119438	0.122101	0.024694	0.204888	0.00627	0.140006	0.373057	0.582278	0.572327	0.516791	0.265363	1	1	0	0	Industrial
6	Vijayawada	01-01-2026 10:00	3409492	HB Colony	16.5206	80.6404	0.150887	0.157543	0.036707	0.136257	0.009117	0.090236	0.487047	0.443038	0.559748	0.479478	0.265363	1	1	0	0	Industrial
7	Vijayawada	01-01-2026 11:00	3409492	HB Colony	16.5206	80.6404	0.185012	0.20579	0.036707	0.172246	0.008347	0.090236	0.528497	0.392405	0.534591	0.401119	0.243017	1	1	0	0	Industrial
8	Vijayawada	01-01-2026 12:00	3409492	HB Colony	16.5206	80.6404	0.100033	0.09872	0.040712	0.189823	0.007471	0.119172	0.57513	0.341772	0.459119	0.337687	0.22905	1	1	0	0	Industrial
9	Vijayawada	01-01-2026 13:00	3409492	HB Colony	16.5206	80.6404	0.125125	0.112637	0.043382	0.225812	0.012513	0.156905	0.61658	0.291139	0.36478	0.350746	0.215084	1	1	0	0	Industrial
10	Vijayawada	01-01-2026 14:00	3409492	HB Colony	16.5206	80.6404	0.11375	0.098534	0.066073	0.244225	0.009117	0.182833	0.632124	0.253165	0.295597	0.393657	0.184358	1	1	0	0	Vehicular
11	Vijayawada	01-01-2026 15:00	3409492	HB Colony	16.5206	80.6404	0.056541	0.042123	0.052725	0.270171	0.013297	0.165471	0.626943	0.253165	0.27044	0.397388	0.178771	1	1	0	0	Vehicular
12	Vijayawada	01-01-2026 16:00	3409492	HB Colony	16.5206	80.6404	0.07193	0.052886	0.045161	0.245899	0.014211	0.155748	0.595855	0.265823	0.283019	0.354478	0.195531	1	1	0	0	Natural
13	Vijayawada	01-01-2026 17:00	3409492	HB Colony	16.5206	80.6404	0.056206	0.039154	0.022469	0.262638	0.01382	0.201352	0.497409	0.35443	0.308176	0.384328	0.284916	1	1	0	0	Natural
14	Vijayawada	01-01-2026 18:00	3409492	HB Colony	16.5206	80.6404	0.061559	0.043793	0.014905	0.274356	0.006087	0.151581	0.362694	0.594937	0.36478	0.660448	0.368715	1	1	0	0	Natural
15	Vijayawada	01-01-2026 19:00	3409492	HB Colony	16.5206	80.6404	0.084644	0.063648	0.014461	0.26766	0.005878	0.150424	0.300518	0.683544	0.433962	0.483209	0.340782	1	1	0	0	Natural
16	Vijayawada	01-01-2026 20:00	3409492	HB Colony	16.5206	80.6404	0.121445	0.095565	0.014461	0.232508	0.005878	0.150424	0.26943	0.721519	0.503145	0.455224	0.343575	1	1	0	0	Industrial
17	Vijayawada	01-01-2026 21:00	3409492	HB Colony	16.5206	80.6404	0.088993	0.067731	0.014461	0.249247	0.005917	0.150424	0.233161	0.78481	0.528302	0.376866	0.329609	1	1	0	0	Natural
18	Vijayawada	01-01-2026 22:00	3409492	HB Colony	16.5206	80.6404	0.061224	0.043793	0.014461	0.232508	0.005865	0.150424	0.207254	0.822785	0.534591	0.253731	0.293296	1	1	0	0	Natural
19	Vijayawada	01-01-2026 23:00	3409492	HB Colony	16.5206	80.6404	0.111408	0.086658	0.014461	0.183127	0.005852	0.150424	0.19171	0.886076	0.528302	0.233209	0.290503	1	1	0	0	Industrial
20	Vijayawada	02-01-2026 00:00	3409492	HB Colony	16.5206	80.6404	0.043158	0.020412	0.016685	0.269334	0.001946	0.150424	0.170984	0.911392	0.509434	0.233209	0.240223	1	1	0	0	Natural
21	Vijayawada	02-01-2026 01:00	3409492	HB Colony	16.5206	80.6404	0.04182	0.019113	0.016685	0.269334	0.00192	0.150424	0.15544	0.924051	0.446541	0.300373	0.173184	1	1	0	0	Natural
22	Vijayawada	02-01-2026 02:00	3409492	HB Colony	16.5206	80.6404	0.041485	0.018742	0.016685	0.269334	0.001894	0.150424	0.139896	0.924051	0.440252	0.283582	0.131285	1	1	0	0	Natural
23	Vijayawada	02-01-2026 03:00	3409492	HB Colony	16.5206	80.6404	0.041485	0.018556	0.016685	0.269334	0.001881	0.150424	0.124352	0.898734	0.440252	0.376866	0.111732	1	1	0	0	Natural
24	Vijayawada	02-01-2026 04:00	3409492	HB Colony	16.5206	80.6404	0.041485	0.018556	0.016685	0.269334	0.00192	0.150424	0.103627	0.898734	0.45283	0.402985	0.106145	1	1	0	0	Natural
25	Vijayawada	02-01-2026 07:00	3409492	HB Colony	16.5206	80.6404	0.115758	0.09705	0.016685	0.269334	0.00192	0.153896	0.134715	0.860759	0.597484	0.408582	0.170391	1	1	0	0	Industrial
26	Vijayawada	02-01-2026 08:00	3409492	HB Colony	16.5206	80.6404	0.065908	0.04435	0.016685	0.269334	0.001907	0.152739	0.248705	0.734177	0.672956	0.455224	0.215084	1	1	0	0	Natural
27	Vijayawada	02-01-2026 09:00	3409492	HB Colony	16.5206	80.6404	0.115089	0.096307	0.016685	0.269334	0.00192	0.152276	0.367876	0.594937	0.691824	0.485075	0.251397	1	1	0	0	Industrial

vijayawada_labelled_dataset

W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ
dist_to_in	dist_to_ro	dist_to_du	hour	day_of_w	day_of_ye	month	is_weeken	is_rush_hour	is_dry_season	season			
2.508861	1.642754	3.531211	6	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	7	3	1	1	0	1	1	1			
2.508861	1.642754	3.531211	8	3	1	1	0	1	1	1			
2.508861	1.642754	3.531211	9	3	1	1	0	1	1	1			
2.508861	1.642754	3.531211	10	3	1	1	0	1	1	1			
2.508861	1.642754	3.531211	11	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	12	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	13	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	14	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	15	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	16	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	17	3	1	1	0	1	1	1			
2.508861	1.642754	3.531211	18	3	1	1	0	1	1	1			
2.508861	1.642754	3.531211	19	3	1	1	0	1	1	1			
2.508861	1.642754	3.531211	20	3	1	1	0	1	1	1			
2.508861	1.642754	3.531211	21	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	22	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	23	3	1	1	0	0	1	1			
2.508861	1.642754	3.531211	0	6	32	2	1	0	1	1			
2.508861	1.642754	3.531211	1	6	32	2	1	0	1	1			
2.508861	1.642754	3.531211	2	6	32	2	1	0	1	1			
2.508861	1.642754	3.531211	3	6	32	2	1	0	1	1			
2.508861	1.642754	3.531211	4	6	32	2	1	0	1	1			
2.508861	1.642754	3.531211	7	6	32	2	1	1	1	1			
2.508861	1.642754	3.531211	8	6	32	2	1	1	1	1			
2.508861	1.642754	3.531211	9	6	32	2	1	1	1	1			

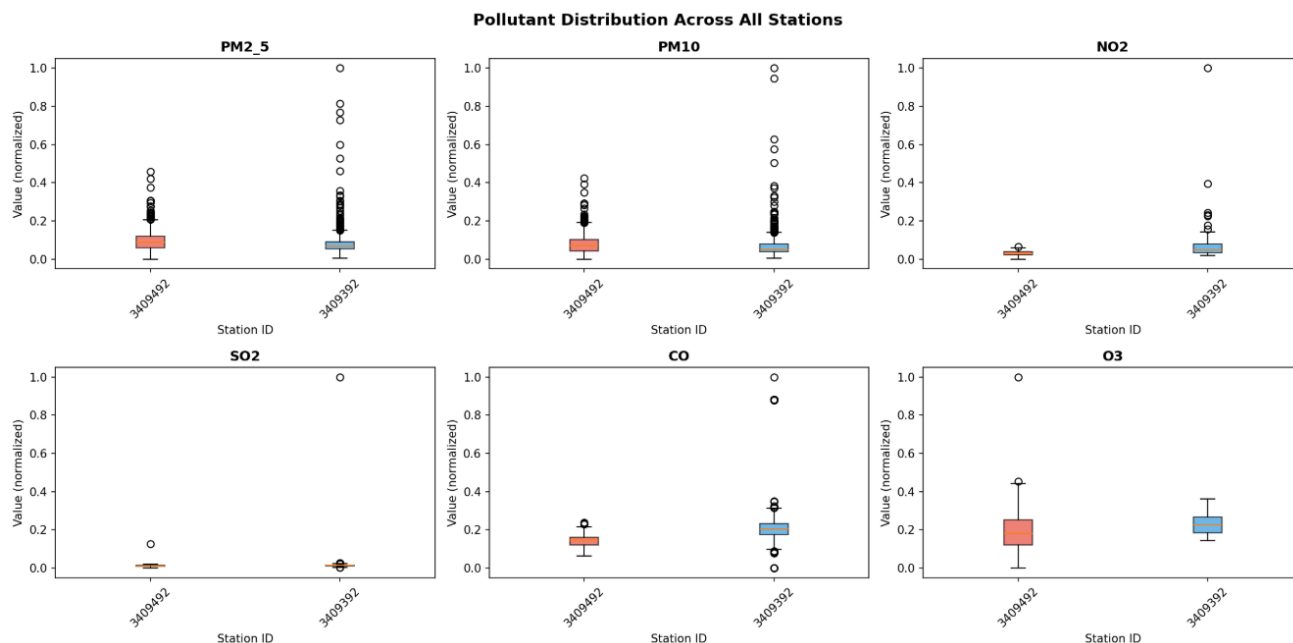
Data analysis of labelled dataset using visualizations:

After assigning pollution source labels to the dataset, exploratory data analysis was performed to understand the **distribution of pollution sources and the relationship between pollutants and environmental factors**. Various visualization techniques were used to identify patterns and trends in the labelled data.

The plots help in understanding **how different pollutants contribute to various pollution sources such as vehicular, industrial, agricultural, and burning activities**.

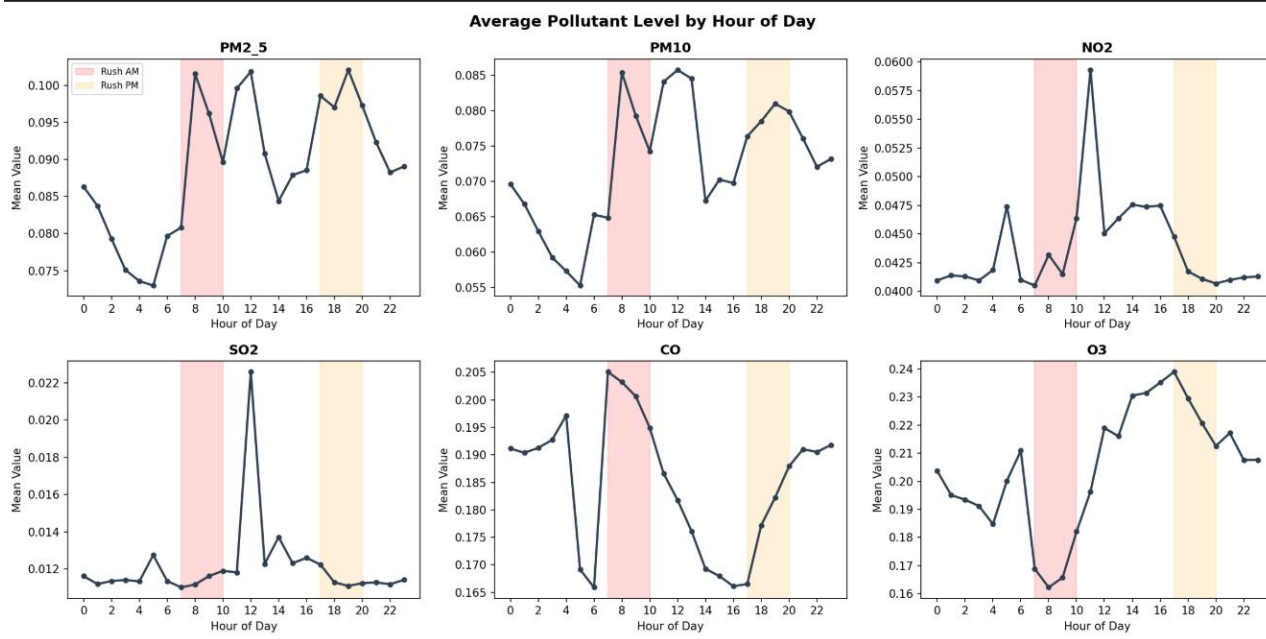
1. Pollutant Distribution Across Stations

This figure shows the distribution of different pollutants such as **PM2.5, PM10, NO2, SO2, CO, and O3** across two monitoring stations. The boxplots represent how the pollutant values vary at each station. The middle line of each box indicates the median value, while the box shows the range where most values fall. The dots outside the box represent outliers or unusually high pollution levels. From the plot, we can see that **PM2.5 and PM10 have larger variation**, indicating fluctuating particulate pollution. Other pollutants like **SO2 and NO2 show lower values with fewer variations**, suggesting relatively stable levels across stations.



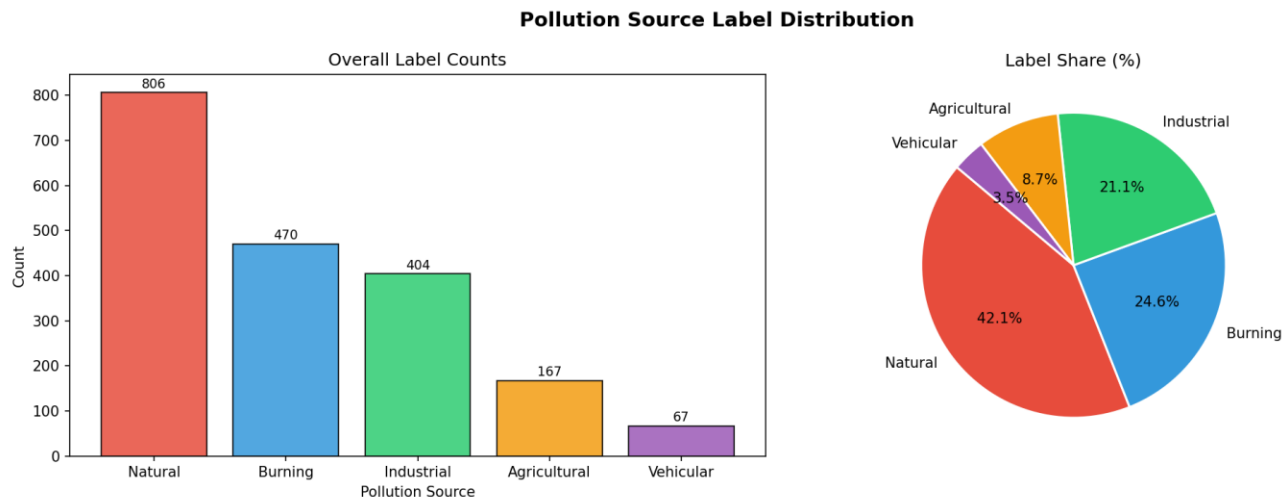
2. Average Pollutant Level by Hour of Day

This figure shows how pollutant levels change throughout the **24 hours of a day**. Each line graph represents the hourly average concentration of a pollutant. The shaded areas highlight **morning rush hours (around 7–10 AM)** and **evening rush hours (around 5–8 PM)**. During these periods, pollutants like **PM2.5, PM10, and CO tend to increase**, likely due to traffic and human activities. Ozone (O3) generally increases during the afternoon due to sunlight and chemical reactions in the atmosphere. This analysis helps understand how pollution varies during different times of the day.



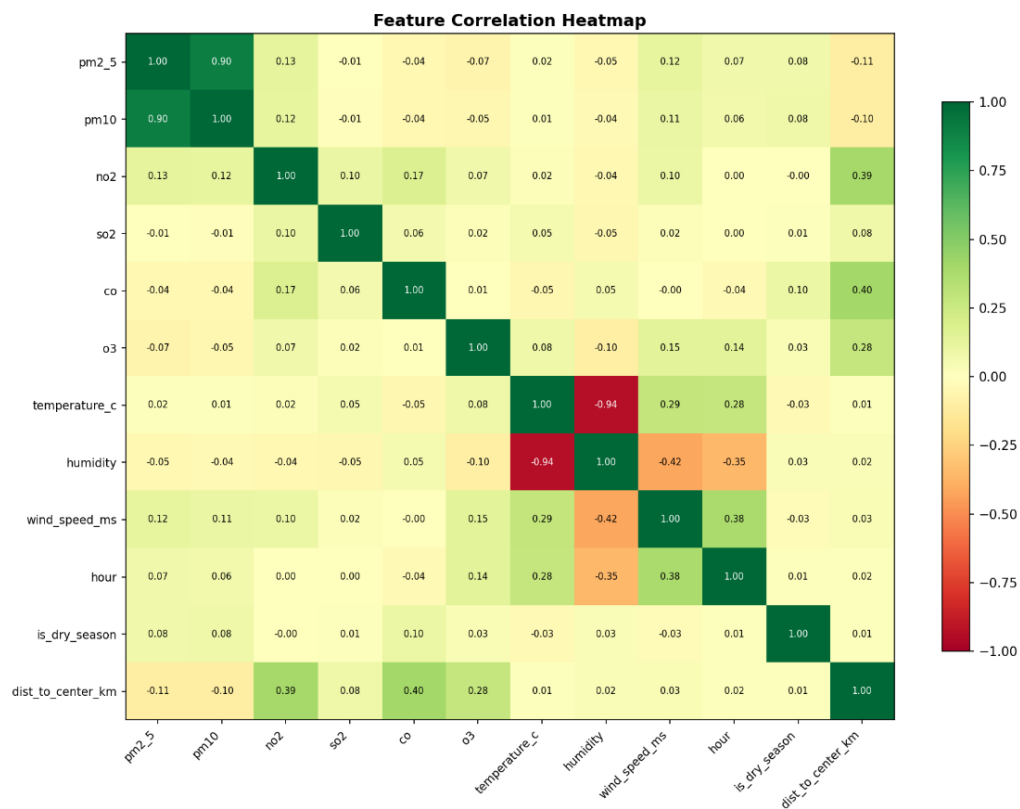
3. Pollution Source Label Distribution

This visualization shows the distribution of different **pollution source categories** in the dataset. The bar chart displays the total count of records for each source, while the pie chart shows the percentage share. The results indicate that **Natural sources contribute the highest share (about 37.5%)**, followed by **Industrial (21.8%)** and **Burning activities (18.4%)**. Agricultural and vehicular sources contribute smaller portions. This distribution helps understand which sources are most responsible for pollution in the dataset.



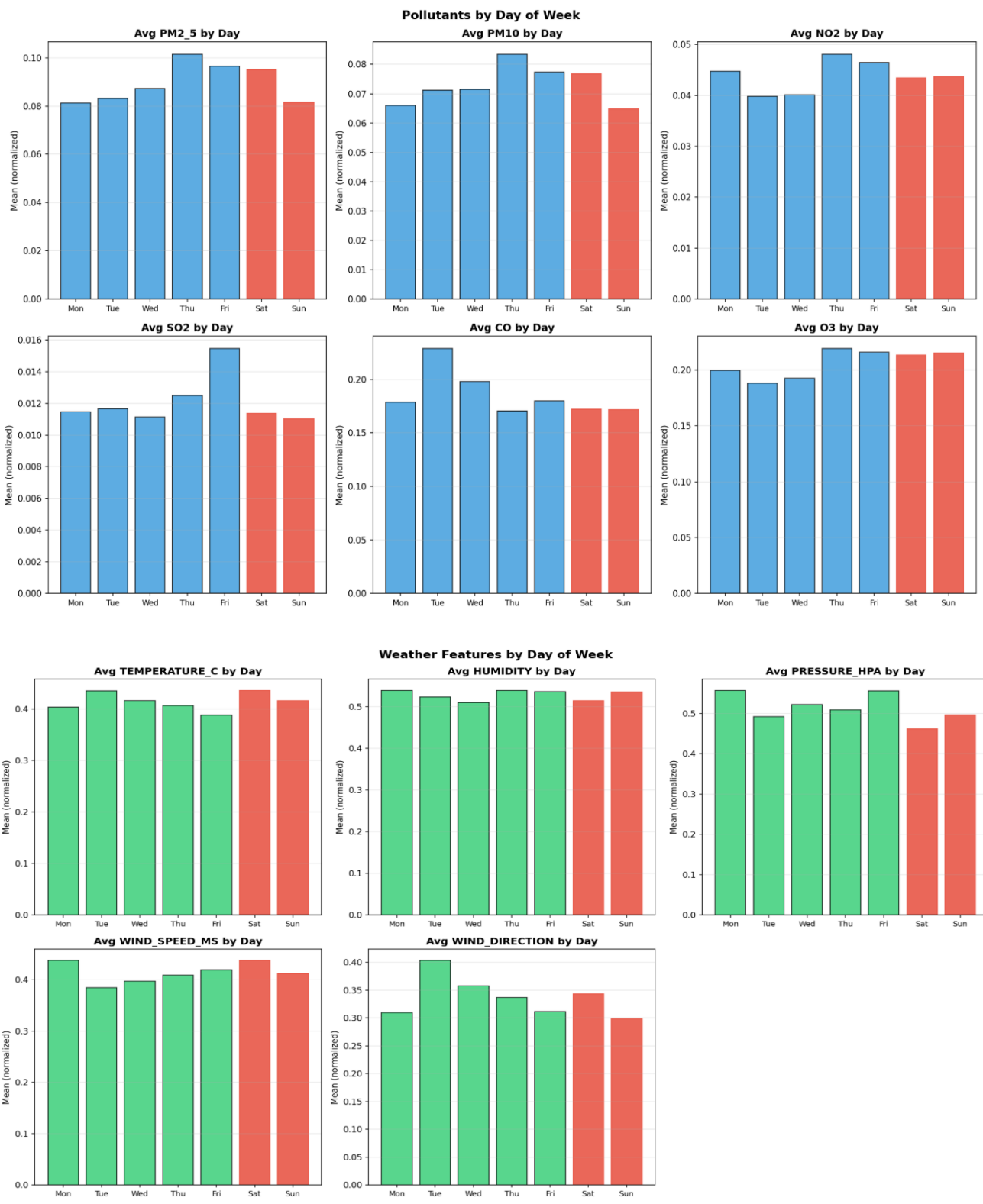
4. Feature Correlation Heatmap

The correlation heatmap illustrates the relationship between different variables such as pollutants and weather conditions. Values range from **-1 to 1**, where values close to 1 indicate strong positive correlation and values close to -1 indicate strong negative correlation. The heatmap shows a strong positive correlation between **PM2.5 and PM10**, meaning they tend to increase together. Temperature and humidity show a strong negative correlation, indicating that when temperature increases, humidity generally decreases. Other relationships are weaker, suggesting that multiple factors influence pollution levels.



5. Pollutants and Weather by Day of Week

This figure shows the **average pollutant levels for each day of the week**. Each subplot represents a pollutant such as PM2.5, PM10, NO2, SO2, CO, and O3. The bars compare weekday values (Monday–Friday) with weekend values (Saturday–Sunday). In many cases, pollutant levels slightly increase during **mid-week due to higher industrial and transportation activity**. Weekend levels are often slightly lower, indicating reduced human activity. This analysis helps identify weekly patterns in pollution levels.



MODULE-4 Model Training and Source Prediction

In this module, the labeled air quality dataset was split into training and test sets using an 80/20 ratio to ensure proper model evaluation. Classification models including Random Forest, XGBoost, Gradient Boosting and Decision Tree were trained using pollutant concentrations (PM2.5, PM10, CO, NO2, SO2, O3), weather parameters (temperature, humidity, pressure, wind speed/direction), as input variables to predict the source of pollution.

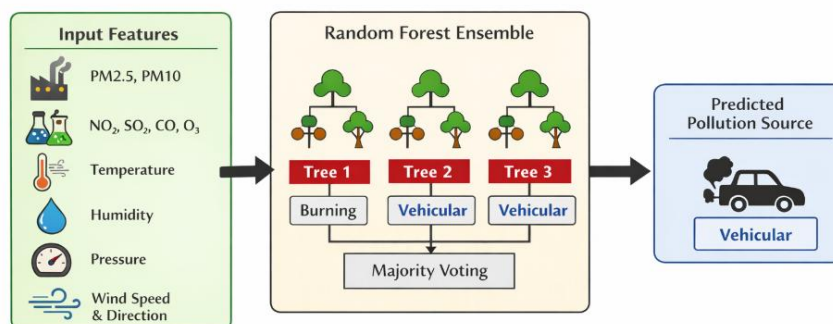
Stratified K-Fold cross-validation was applied to maintain class distribution during training and validate model performance robustly. Hyperparameters for each model were optimized using GridSearchCV and RandomizedSearchCV to improve predictive performance. Model evaluation was carried out using accuracy, precision, recall, F1-score, and confusion matrix, providing a comprehensive assessment of their effectiveness.

Finally, the best-performing model was exported using joblib/pickle for integration into the pollution monitoring dashboard, enabling real-time source prediction from environmental sensor data.

Model-1 Random Forest:

Random Forest is a supervised machine learning algorithm widely used for classification tasks. It is based on an ensemble learning approach where multiple decision trees are created and their predictions are combined to produce a more reliable and accurate result. By aggregating the outputs of many trees, Random Forest reduces the risk of overfitting and improves the model's generalization capability.

Random Forest for Pollution Source Detection



Implementation:

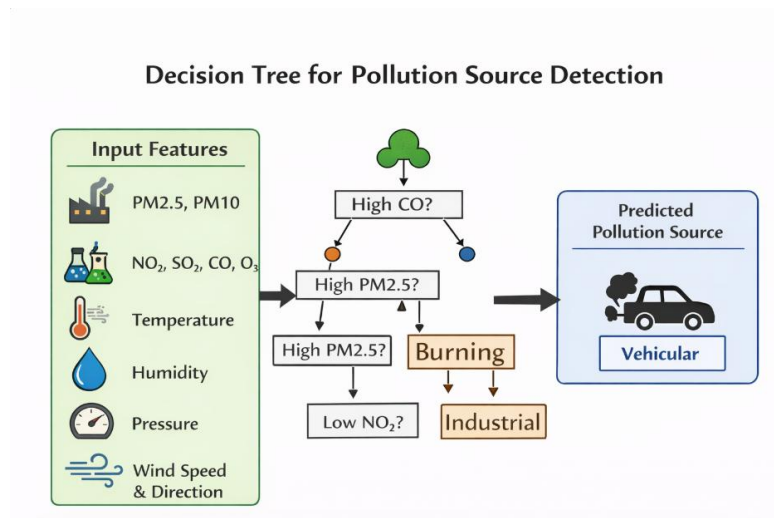
The pollution source prediction module uses a labelled dataset of 1914 samples with features including pollutant concentrations weather conditions and the target variable pollution_source with five categories (Industrial,

Burning, Natural, Agricultural, Vehicular). The dataset was split into training and test sets (80/20) with stratification, and SMOTE oversampling was applied to balance the classes.

A Random Forest model was trained, and at first it gave the 95.4 % for training accuracy and 83.03 % for test accuracy and hyperparameters were optimized using GridSearchCV. Model evaluation included accuracy, precision, recall, F1-score, and confusion matrices, along with cross-validation using Stratified K-Fold (5 folds) to ensure robust performance. The tuned model achieved a train accuracy of **99.1%** and test accuracy of **88.51%**, with an average cross-validated F1-score of 0.905. Feature importance analysis identified no2, co, pm10, pm2_5, and o3 as the most influential predictors. Visualizations including decision tree plots, feature importance charts, and confusion matrices were generated, and the trained model was exported using joblib for integration into a pollution monitoring dashboard, providing a reliable tool for identifying pollution sources based on environmental data.

Model-2 Decision Tree:

A Decision Tree is a supervised machine learning algorithm used for classification and regression problems. It works like a tree structure, where data is divided step-by-step based on conditions to make a final prediction.



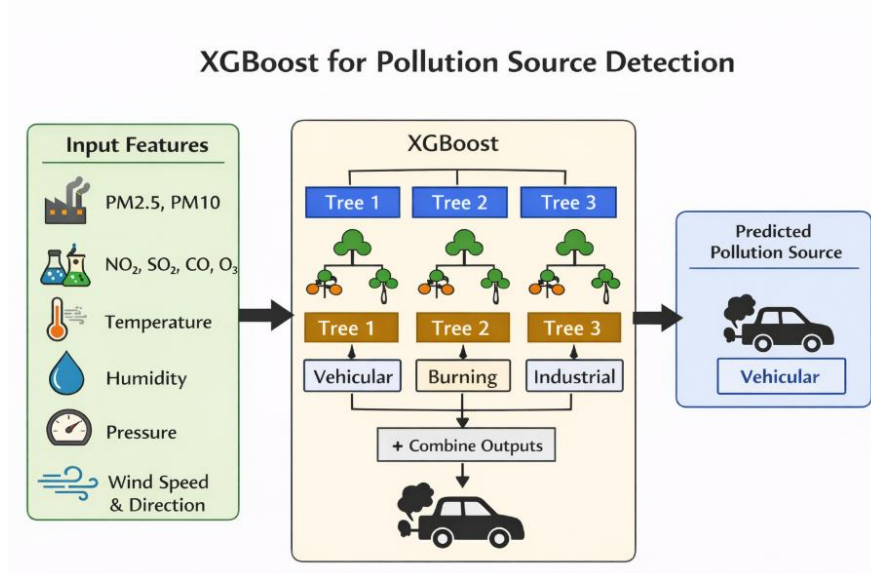
Implementation

The Decision Tree model for pollution source prediction was trained on a labeled dataset of 1914 samples, using pollutant concentrations weather features and pollution_source as the target variable with five classes. The dataset was split into training and test sets (80/20) with stratification, and SMOTE oversampling was applied to balance the classes.

A base Decision Tree achieved 100% train accuracy and 88.25% test accuracy. After hyperparameter tuning with RandomizedSearchCV, the model improved to **95.27%** train accuracy and **90.08%** test accuracy, with cross-validated F1-score averaging 0.915. The features importance analysis highlighted PM10, CO, NO2, and PM2.5 as the most influential variables in determining pollution sources, reflecting the significant role of particulate matter and gaseous pollutants in air quality assessment. Confusion matrices, tree visualizations, and explicit decision rules were generated, identifying key feature splits for pollution classification. The final model was saved using joblib for integration into a monitoring dashboard.

Model-3 XG Boost:

XGBoost (Extreme Gradient Boosting) **is a** powerful machine learning algorithm **used for** classification and regression problems. It is based on the concept of Gradient Boosting, where multiple **decision trees** are built sequentially to improve prediction accuracy. In XGBoost, **each new tree** learns from the errors of the previous trees and tries to correct them. By combining many weak trees, the model produces a strong and accurate prediction.



Implementation

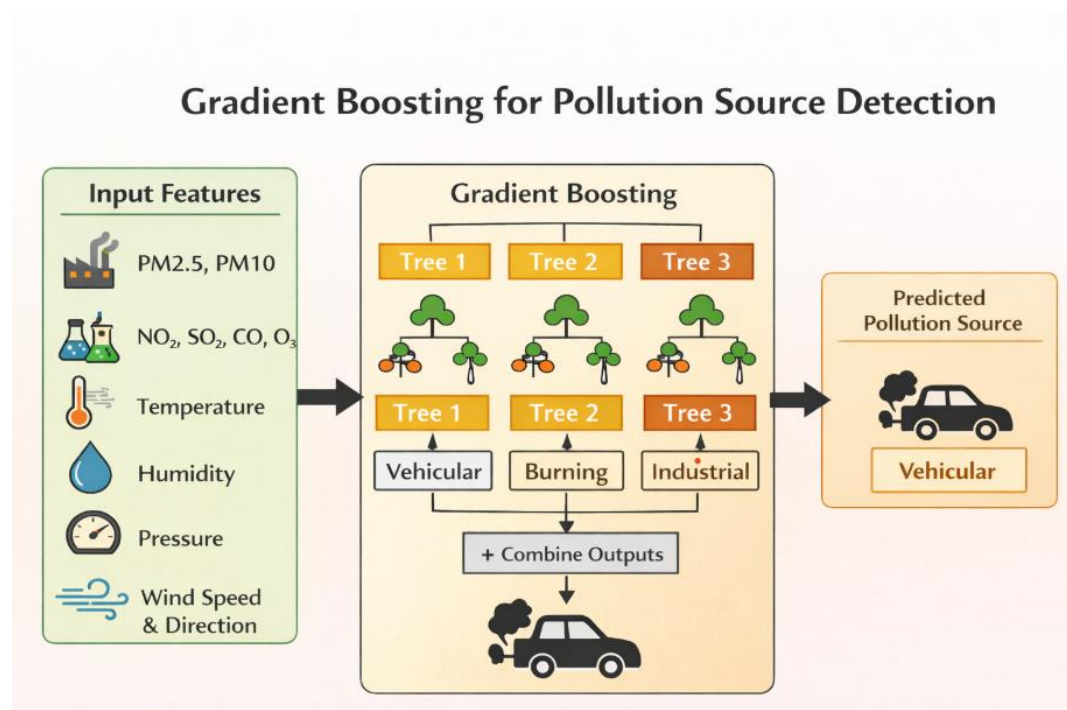
The pollution source prediction module uses a labelled dataset of 1914 samples with features including pollutant concentrations, weather conditions and the target variable pollution_source with five categories (Industrial,

Burning, Natural, Agricultural, Vehicular). The dataset was split into training and test sets (80/20) with stratification, and SMOTE oversampling was applied to balance the classes.

An XGBoost classifier was trained, at first it gives 94.74 % for training accuracy and 89.56 % for test accuracy after that hyperparameters were optimized using GridSearchCV. Model evaluation included accuracy, precision, recall, F1-score, and confusion matrices, along with cross-validation using Stratified K-Fold (5 folds) to ensure robust performance. The tuned model achieved a train accuracy of **97.54%** and test accuracy of **91.64%**, with an average cross-validated F1-score of 0.942. Feature importance analysis identified pm2_5, pm10, co, no2, and o3 as the most influential predictors. Visualizations including feature importance charts and confusion matrices were generated, and the trained model was exported using joblib for integration into a pollution monitoring dashboard, providing a reliable tool for identifying pollution sources based on environmental data.

Model-4 Gradient Boosting:

Gradient Boosting is an ensemble learning method that builds models sequentially. Unlike Random Forest, where trees are built independently in parallel, Gradient Boosting builds one tree at a time. Each new tree is specifically trained to predict and correct the errors (residuals) made by the previous trees.



Implementation

The pollution source prediction module uses a labelled dataset of 1914 samples with features including pollutant concentrations and weather conditions with the target variable `pollution_source` categorized into five classes (Industrial, Burning, Natural, Agricultural, Vehicular). The dataset was balanced using SMOTE and split into training and test sets (80/20) with stratification.

A Gradient Boosting Classifier was trained and evaluated, achieving a train accuracy of 99.78% and test accuracy of 96.89% before tuning. Hyperparameter tuning with GridSearchCV improved model performance, resulting in a train accuracy of **96.64%** and test accuracy of **93.16%** after tuning, along with an average cross-validated F1-score of 0.939. Feature importance analysis identified `pm2_5`, `pm10`, `no2`, `co`, and `o3` as the most influential predictors. Confusion matrix and feature importance plots were generated, and the trained model was exported using joblib, providing a robust tool for identifying pollution sources from environmental data.

RESULTS

The dataset used for this study contained **1914 samples and 33 attributes**. From these, **11 important environmental features** were selected for model training, including pollutant concentrations (**pm2_5, pm10, no2, o3, so2, co**) and meteorological factors (**temperature, humidity, pressure, wind speed, wind direction**). The dataset had a **Gini Index of 0.709**, indicating a moderate level of class diversity.

To address class imbalance, **SMOTE (Synthetic Minority Oversampling Technique)** was applied, resulting in a balanced dataset with **643 samples for each pollution source category**: Agricultural, Burning, Industrial, Natural, and Vehicular.

Several machine learning models were trained and evaluated, including **Decision Tree, Random Forest, XGBoost, and Gradient Boosting**. Hyperparameter tuning was performed to improve model performance.

The **Decision Tree model** achieved a test accuracy of **88.25% before tuning** and improved to **90.08% after tuning**, with an average cross-validation F1-score of **0.915**, indicating stable and reliable performance.

The **Random Forest model** initially achieved **83.03% accuracy**, which improved to **88.51% after hyperparameter tuning**. The model showed good predictive capability with an **average cross-validation F1-score of 0.905**. Feature importance analysis revealed that **NO₂, CO, PM10, and PM2.5** were the most influential variables for predicting pollution sources.

The **XGBoost model** produced **89.56% accuracy before tuning**, which increased to **91.64% after tuning**, with a strong **average cross-validation F1-score of 0.942**, indicating improved generalization compared to tree-based ensemble models.

The **Gradient Boosting model** achieved the highest performance among all models. It reached **96.89% accuracy before tuning** and **93.16% after tuning**, with an **average cross-validation F1-score of 0.939**. The results demonstrate that Gradient Boosting effectively captures complex relationships between environmental variables and pollution sources.

Overall, the experimental results show that **ensemble learning methods outperform single decision tree models**, with **Gradient Boosting providing the best predictive accuracy for pollution source classification**. The findings also highlight the importance of pollutant concentration variables such as **NO₂, CO, PM10, and PM2.5** in identifying pollution sources.

CONCLUSION

This project developed a machine learning framework to **identify pollution sources using environmental and meteorological data**. A dataset containing **1914 records and 33 attributes** was processed, and **11 key features** related to air pollutants and weather conditions were used for model training.

To handle class imbalance, **SMOTE** was applied, resulting in a balanced dataset for the five pollution source categories: **Agricultural, Burning, Industrial, Natural, and Vehicular**.

Multiple machine learning models, including **Decision Tree, Random Forest, XGBoost, and Gradient Boosting**, were trained and evaluated.

Hyperparameter tuning and cross-validation were used to improve model reliability and performance. Among the tested models, **Gradient Boosting achieved the highest predictive accuracy**, demonstrating strong capability in capturing complex relationships between environmental factors and pollution sources.

The results show that **ensemble learning techniques provide better performance compared to single tree models**, and that pollutant variables such as **NO₂, CO, PM₁₀, and PM_{2.5}** play a major role in determining pollution sources.

Overall, the proposed approach can help support **environmental monitoring and decision-making by accurately classifying pollution sources based on sensor data**.

FUTURE SCOPE

Although the EnviroScan system achieved promising results, several improvements can be implemented in the future to enhance the system further.

1. Real-Time Pollution Monitoring

The system can be integrated with real-time pollution sensors to continuously monitor air quality and provide live pollution source predictions.

2. Expansion to More Locations

Currently, the project focuses on selected areas of Vijayawada. In the future, the system can be expanded to analyze pollution patterns across multiple cities.

3. Integration with Satellite Data

Satellite-based environmental data can be incorporated to improve large-scale pollution monitoring and provide better spatial coverage.

4. Mobile and Web Applications

A user-friendly mobile or web application can be developed to allow citizens and authorities to monitor pollution levels easily.

5. Advanced Machine Learning Techniques

Future work can explore deep learning models and advanced data analytics techniques to improve prediction accuracy and handle larger dataset

REFERENCES

<https://docs.openaq.org/>

<https://openweathermap.org/api>

<https://docs.python.org/3/>